

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	Assessment:	Assessment Tool 1 – Application Based Questions
Weightage:	40% of CIA		
CO1 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	K .NAVEEN KUMAR REDDY	Reg. No.:	192425203
Section / Batch:		Date:	

Code of Conduct

I K NAVEEN KUMAR REDDY(192425203) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: NAVEEN

Instructions

- Develop a modular and efficient Python program that satisfies all the functional requirements specified in the problem statement.
- Demonstrate the correctness of the implementation using suitable test cases and justify the output obtained.
- Follow good programming practices, including meaningful variable names, proper code indentation, comments, and error handling wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Information Extraction	Regex-Based Information Extraction	1
Pattern Matching	Regex Pattern Matching	2
Regular Expression Patterns	Regex-Based Input Validation	3

Section 1: Regular Expressions – Resume Information Extraction

1. A multinational recruitment company receives thousands of resumes every day in text format. The HR department requires an automated system to extract candidate information for shortlisting.

Design and implement a Python-based resume information extraction system using Regular Expressions that performs the following tasks:

Extract the candidate's name.

Identify email addresses.

Extract mobile numbers.

Detect technical skills (Python, Java, SQL, Machine Learning, NLP).

Extract years of experience.

Generate a structured summary of the candidate's profile.

Display candidates who satisfy the minimum eligibility criteria (minimum 2 years of experience and Python skill).

Section 2: Regular Expressions – Product Search System

2. An e-commerce company plans to enhance its product search engine to improve customer experience through flexible keyword matching.

Design and implement a Python-based product search system using Regular Expressions that performs the following tasks:

Search products using an exact keyword.

Perform prefix-based searches.

Perform suffix-based searches.

Search using partial keywords.

Perform case-insensitive searches.

Display all matching product names.

Generate a report showing the total number of matching products for each search.

Section 3: Regular Expressions – University Registration System

A university is developing an automated online registration portal to verify student information before course enrollment.

Design and implement a Python-based validation system using Regular Expressions that performs the following tasks:

Validate the student register number.

Validate the institutional email address.

Validate the course code.

Validate the semester information.

Validate the student's mobile number.

Display appropriate validation messages for each field.

Generate a final registration status report indicating whether the registration is successful.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Q. No. / Criteria	Understanding of Problem Context	Application of Concepts	Problem-Solving Approach	Accuracy of Solution	Clarity	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

COMPLETED APPLICATION-BASED ANSWERS

Natural Language Processing (DSA03) Assessment Tool 1 – Application Based Questions

Question 1 – Resume Information Extraction

Application: A multinational recruitment company can automate resume screening by using Regular Expressions (Regex) to extract candidate details and shortlist candidates who have at least 2 years of experience and Python skill.

AIM

To develop a modular Python program using Regular Expressions to extract resume information and identify eligible candidates.

ALGORITHM

1. Store resume text for multiple candidates.
2. Use Regex patterns to extract name, email, mobile number, technical skills, and years of experience.
3. Create a structured profile for each candidate.
4. Check whether experience is at least 2 years and Python is present in the skills.
5. Display the extracted profile and eligibility status.

PYTHON PROGRAM

```
import re

resumes = [
    """Name: Arun Kumar
    Email: arun.kumar@gmail.com
    Mobile: 9876543210
    Skills: Python, SQL, NLP
    Experience: 3 years""",

    """Name: Priya Sharma
    Email: priya.sharma@gmail.com
    Mobile: 9123456780
    Skills: Java, SQL
    Experience: 1 year""",

    """Name: Rahul Raj
    Email: rahul.raj@gmail.com
    Mobile: 9988776655
    Skills: Python, Java, Machine Learning
    Experience: 5 years""",
]

skill_pattern = re.compile(
    r'\b(Python|Java|SQL|Machine Learning|NLP)\b', re.I
)

for resume in resumes:
    name = re.search(r'Name:\s*(.+)', resume)
    email = re.search(r'[\w.-]+@[\w.-]+\.\w+', resume)
    mobile = re.search(r'\b\d{10}\b', resume)
    exp = re.search(r'Experience:\s*(\d+)\s*years?', resume, re.I)
    skills = skill_pattern.findall(resume)

    name = name.group(1) if name else "Not found"
    email = email.group(0) if email else "Not found"
    mobile = mobile.group(0) if mobile else "Not found"
    years = int(exp.group(1)) if exp else 0

    print("\nCandidate Profile")
    print("Name:", name)
    print("Email:", email)
    print("Mobile:", mobile)
    print("Skills:", ", ".join(dict.fromkeys(skills)))
    print("Experience:", years, "years")

    if years >= 2 and any(s.lower() == "python" for s in skills):
        print("Eligibility: ELIGIBLE")
    else:
        print("Eligibility: NOT ELIGIBLE")
```

SAMPLE OUTPUT

Candidate Profile

Name: Arun Kumar
Email: arun.kumar@gmail.com
Mobile: 9876543210
Skills: Python, SQL, NLP
Experience: 3 years
Eligibility: ELIGIBLE

Candidate Profile

Name: Priya Sharma
Email: priya.sharma@gmail.com
Mobile: 9123456780
Skills: Java, SQL
Experience: 1 years
Eligibility: NOT ELIGIBLE

Candidate Profile

Name: Rahul Raj
Email: rahul.raj@gmail.com
Mobile: 9988776655
Skills: Python, Java, Machine Learning
Experience: 5 years
Eligibility: ELIGIBLE

APPLICATION / JUSTIFICATION

Regex is applied directly to the recruitment problem. Email and mobile patterns identify contact details, the skill pattern detects required technical skills, and the eligibility rule combines minimum experience with Python skill. This reduces manual screening and creates a structured shortlist.

RESULT

The Python Regex system extracts resume information and identifies candidates who satisfy the specified eligibility criteria.

Question 2 – Product Search System

Application: An e-commerce company can use Regex to provide flexible product searches, allowing customers to search by exact, prefix, suffix, partial, and case-insensitive keywords.

AIM

To develop a Python-based Regex product search system that displays matching products and reports the number of matches.

ALGORITHM

1. Store product names in a list.
2. Create Regex patterns for exact, prefix, suffix, and partial searches.
3. Use case-insensitive matching so upper- and lower-case keywords are treated equally.
4. Compare the pattern with every product name.
5. Display all matches and the total number of matching products.

PYTHON PROGRAM

```
import re

products = [
    "Python Programming Book",
    "Python Laptop",
    "Java Programming Book",
    "SQL Database Guide",
    "Machine Learning Course",
    "NLP Handbook",
    "Gaming Laptop",
    "Python Notebook"
]

def search_products(keyword, mode):
    if mode == "exact":
        pattern = rf"^{re.escape(keyword)}$"
    elif mode == "prefix":
        pattern = rf"^{re.escape(keyword)}"
    elif mode == "suffix":
        pattern = rf"{re.escape(keyword)}$"
    elif mode == "partial":
```

```

        pattern = re.escape(keyword)
    else:
        print("Invalid search mode")
        return

matches = [p for p in products if re.search(pattern, p, re.I)]

print("\nSearch type:", mode)
print("Keyword:", keyword)
print("Matching products:")
for p in matches:
    print("-", p)
print("Total matches:", len(matches))

search_products("Python", "prefix")
search_products("Laptop", "suffix")
search_products("programming", "partial")
search_products("SQL Database Guide", "exact")
search_products("PYTHON", "partial")

```

SAMPLE OUTPUT

Search type: prefix
 Keyword: Python
 Matching products:
 - Python Programming Book
 - Python Laptop
 - Python Notebook
 Total matches: 3

Search type: suffix
 Keyword: Laptop
 Matching products:
 - Python Laptop
 - Gaming Laptop
 Total matches: 2

Search type: partial
 Keyword: programming
 Matching products:
 - Python Programming Book
 - Java Programming Book
 Total matches: 2

Search type: exact
 Keyword: SQL Database Guide
 Matching products:
 - SQL Database Guide
 Total matches: 1

Search type: partial
 Keyword: PYTHON
 Matching products:
 - Python Programming Book
 - Python Laptop
 - Python Notebook
 Total matches: 3

APPLICATION / JUSTIFICATION

The system models an e-commerce search feature. ^ supports prefix matching, \$ supports suffix matching, re.I provides case-insensitive matching, and partial matching allows customers to enter only part of a product name. The count provides a useful search report.

RESULT

The Regex product search system performs the required search operations and generates the number of matching products.

Question 3 – University Registration System

Application: A university registration portal can validate student information before course enrollment. Regex checks whether each field follows the required format and prevents invalid registration data.

AIM

To implement a Python Regex validation system for register number, institutional email, course code, semester, and mobile number.

ALGORITHM

1. Define Regex patterns for each registration field.
2. Read the student's register number, email, course code, semester, and mobile number.
3. Match each input against its corresponding pattern.
4. Display a validation message for every field.
5. If all fields are valid, display successful registration; otherwise display registration failed.

PYTHON PROGRAM

```
import re

def validate(pattern, value):
    return bool(re.fullmatch(pattern, value))

register_no = input("Register Number: ")
email = input("Institutional Email: ")
course_code = input("Course Code: ")
semester = input("Semester: ")
mobile = input("Mobile Number: ")

tests = {
    "Register Number": (r"\d{2}[A-Z]{2}\d{4}", register_no),
    "Institutional Email": (r"[\w.-]+@[ \w.-]*college\.edu", email),
    "Course Code": (r"[A-Z]{2,4}\d{3,4}", course_code),
    "Semester": (r"([1-8]|1-8)(?:st|nd|rd|th))", semester),
    "Mobile Number": (r"[6-9]\d{9}", mobile)
}

all_valid = True

for field, (pattern, value) in tests.items():
    if validate(pattern, value):
        print(field, ": VALID")
    else:
        print(field, ": INVALID")
        all_valid = False

print("\nFinal Registration Status:",
      "SUCCESSFUL" if all_valid else "FAILED")
```

SAMPLE INPUT

Register Number: 24AI1234
Institutional Email: student@college.edu
Course Code: DSA03
Semester: 3
Mobile Number: 9876543210

SAMPLE OUTPUT

Register Number : VALID
Institutional Email : VALID
Course Code : VALID
Semester : VALID
Mobile Number : VALID

Final Registration Status: SUCCESSFUL

APPLICATION / JUSTIFICATION

Regex validation is appropriate for a registration portal because each field follows a predictable format. `fullmatch()` checks the complete input against its pattern. Field-level messages help students correct errors, while the final status prevents invalid registration from proceeding.

RESULT

The Regex validation system verifies the required student details and produces a final registration status.

RUBRIC COVERAGE CHECKLIST

Criterion	Weightage	Coverage in Completed Answers
Understanding of Problem Context	20%	Each real-world scenario and all specified functional requirements are addressed.
Application of Concepts	30%	Regex is applied to recruitment, e-commerce search, and university registration.
Problem-Solving Approach	20%	Algorithms, patterns, conditions, functions, and decision rules are provided.
Accuracy of Solution	20%	Python programs include test data, expected outputs, and required logic.
Clarity	10%	Each question is organized into AIM, Algorithm, Program, Output, Application/Justification, and Result.

Important note: The assessment specifies validation tasks but does not provide institution-specific formats for register numbers, email domains, or course codes. The Question 3 patterns are therefore demonstration formats and should be changed if your college uses different formats.